Chapter 7

Load Balancer & Reverse Proxy

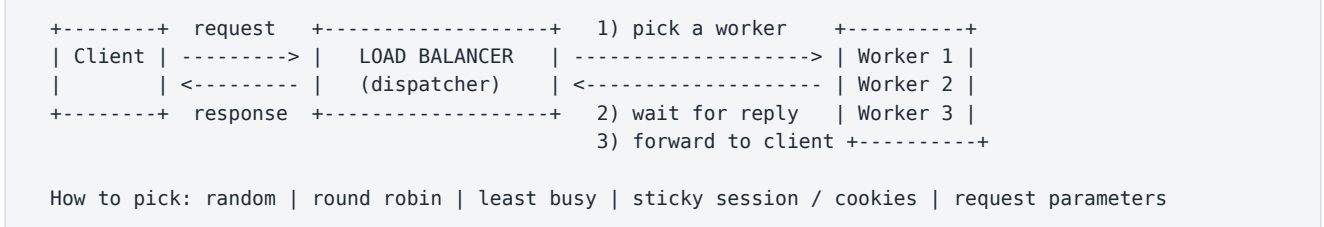
Why this chapter matters: Chapter 5 said “scale out with many stateless servers”. Something must then **spread the traffic**, **notice dead servers** and give clients **one stable address**. That something is the load balancer — usually working together with a reverse proxy. It is on the path of every request, so it influences latency, availability, security and cost. Almost every system design diagram has one.

⚡ **QUICK RECALL — Chapter map**

- 7.1 What a load balancer does → 7.2 Types & placement → 7.3 Layer 4 vs Layer 7 → 7.4 Balancing algorithms
- 7.5 Health checks, draining & failure handling → 7.6 Sessions & sticky routing → 7.7 TLS, connections & client IP
- 7.8 Reverse proxy (and API gateway) → 7.9 Making the LB itself highly available → 7.10 Horizontal scaling, autoscaling & capacity math
- 7.11 Disadvantages & pitfalls → 7.12 Worked examples → 7.13 Practice, Q&A, cheat sheet

7.1 What a load balancer does

A **load balancer (LB)** distributes incoming client requests across computing resources such as application servers and databases. In each case, the load balancer **returns the response from the computing resource to the appropriate client**. In the dispatcher picture from your notes it performs three steps:



Job	Meaning	Benefit
Distribute	Spread requests across a pool of identical servers	Prevents overloading resources; more total throughput
Detect & avoid failure	Health checks remove unhealthy servers from rotation	Prevents requests from going to unhealthy servers → availability
Abstract	Clients see one address (VIP / DNS name); servers can be added, removed, upgraded behind it	Helps eliminate a single point of failure at the server tier; zero-downtime deploys
Offload (L7 / proxy mode)	TLS termination, compression, caching, rate limiting, auth	Backends do less work (7.7–7.8)

✎ **CORRECTION TO YOUR ORIGINAL NOTE — Detail added to the dispatcher figure**

- The figure shows the response flowing *back through* the LB. That is true for proxy-style LBs (all L7, most L4). Some L4 designs use **Direct Server Return (DSR)**, where the server replies straight to the client and the response bypasses the LB — useful when responses are huge compared with requests (7.3).

7.2 Types of load balancers & where they sit

Type	Examples	Pros	Cons
Hardware appliance	F5 BIG-IP, Citrix ADC	Very high throughput, specialised features, vendor support	**Expensive**, capacity bought upfront, slower to change, vendor lock-in
Software	**HAProxy, NGINX, Envoy**, Traefik, Caddy	Cheap, flexible, scriptable, runs anywhere (VMs, containers), version-controlled config	You run, patch, scale and make it highly available
Cloud managed	AWS ALB (L7) / NLB (L4) / Gateway LB, GCP Cloud Load Balancing, Azure Load Balancer / Application Gateway	Multi-AZ and auto-scaling by default, integrated with autoscaling, certificates, WAF	Less control, per-hour + per-LCU/GB cost, provider limits
Client-side	gRPC client LB, Netflix Ribbon-style, service discovery (Consul, Eureka)	No extra hop; client picks a healthy instance from a registry	Logic in every client/language; harder to change centrally
Service mesh sidecar	Envoy in Istio/Linkerd	Uniform LB, retries, mTLS between all services	Operational complexity (Chapter 8)
DNS-based	Route 53, Cloudflare (Chapter 6)	Global steering between regions	Cached answers, no load awareness

Where load balancers are placed

```
Tier 1:  Internet -> DNS / CDN -> Edge LB (L4/L7) -> Web / API servers
Tier 2:  Web / API servers -> Internal LB (or service mesh) -> Application services
Tier 3:  Application services -> DB proxy / connection pool -> DB primary + read replicas
```

A real system typically has several LB layers: a public edge LB, internal LBs between service tiers (or a mesh), and proxies/poolers in front of databases and caches (read/write splitting, connection pooling).

7.3 Layer 4 vs Layer 7 load balancing

Load balancers can route traffic based on various metrics, including random, least loaded, session/cookies, round robin or weighted round robin, and the OSI layer they inspect (Chapter 3).

	Layer 4 (transport)	Layer 7 (application)
Looks at	Info at the **transport layer**: source/destination IP addresses and ports in the header — **not the contents of the packet**	The **application layer**: HTTP method, URL path, host, headers, cookies, message body
How it works	Forwards network packets to and from the upstream server, performing **Network Address Translation (NAT)**. A connection is mapped to one backend	**Terminates** the client's network connection, reads the message, makes a load-balancing decision, then **opens a (pooled) connection to the selected server**
Speed & cost	**Less time and computing resources**; millions of connections; very low latency	More CPU/memory (parses protocol, often decrypts TLS); modest impact on modern commodity hardware
Flexibility	Low: same decision for the whole connection	**High**: per-request routing, rewriting, redirects, retries, caching, auth, WAF
Protocols	Any TCP/UDP (databases, MQTT, game servers, TLS passthrough)	HTTP/1.1, HTTP/2, gRPC, WebSocket (protocol-aware)
Examples	AWS NLB, HAProxy (tcp mode), LVS/IPVS, Maglev-style	AWS ALB, NGINX, HAProxy (http mode), Envoy

Content-based routing is the superpower of Layer 7. For example, a Layer 7 load balancer can direct video traffic to servers that host videos while directing more sensitive user billing traffic to security-hardened servers:

Rule (host / path / header)	Target pool
`api.example.com/v1/*`	API servers (stateless, autoscaled)
`/static/*`, `/images/*`	Static/media servers or object storage + CDN
`/video/*`	Video-optimised hosts (big disks, high bandwidth)
`/billing/*`	Hardened, isolated pool (stricter security group, extra WAF rules)
`Upgrade: websocket`	WebSocket gateway pool (long-lived connections, Chapter 4)
`X-Canary:` 1 or 5% of traffic	Canary release pool (weighted routing)

◆ **SENIOR INSIGHT** — **L4 modes and the common two-tier layout**

- **NAT mode**: LB rewrites addresses, replies return through it. **DSR (direct server return)**: LB rewrites only the MAC and the server answers the client directly — high throughput for large responses, but servers need the VIP configured and the LB can't see responses. **IP-in-IP / tunnelling** variants allow servers on other subnets.
- Very large sites run **L4 in front of L7**: a fleet of fast L4 balancers (spread over ECMP routers) distributes connections to a fleet of L7 proxies, which route requests to application servers.
- **TLS passthrough** (L4) keeps traffic encrypted end to end but gives up L7 features; **TLS termination** (L7) enables them (7.7).

7.4 Load-balancing algorithms

Algorithm	How it picks	Good for	Watch out
Random	Uniformly random server	Simple, stateless LBs	Short-term imbalance
Round robin	Servers in turn: 1, 2, 3, 1, 2, 3…	Identical servers, similar request costs	Ignores load and request size; heavy and light requests can pile up unevenly
Weighted round robin	Like RR but a server with weight 3 gets 3× the turns	Mixed hardware, canary (5%), gradual ramp-up	Weights are static
Least connections	Server with the fewest active connections	Variable request duration, long-lived connections	Counts connections, not CPU; a fresh server can be flooded (use slow start)
Weighted least connections	Least connections ÷ weight	Heterogeneous fleets with variable requests	Needs sensible weights
Least response time	Fastest recent response + fewest connections	Latency-sensitive pools	Measurement noise; herd toward the 'fast' server
IP hash / consistent hash	Hash of client IP, user id, URL or header decides the server	**Session affinity**, **cache locality** (same URL → same cache node), sharded backends	Uneven if keys are skewed; plain modulo hashing remaps most keys when servers change → use **consistent hashing** (Chapter 14)
Power of two choices (P2C)	Pick 2 random servers, send to the less loaded one	Large fleets and distributed LBs (Envoy, Nginx+)	Needs load info (conn counts); excellent balance with tiny cost
Resource-based / agent	Backends report CPU/memory/queue depth	Very uneven work	Extra protocol and lag in reported data

Tiny sketch (Node-style) of three algorithms

```
const servers = [{ id: 'A', conns: 0 }, { id: 'B', conns: 0 }, { id: 'C', conns: 0 }];
let i = 0;

const roundRobin = () => servers[i++ % servers.length];
const leastConn  = () => servers.reduce((a, b) => (b.conns < a.conns ? b : a));

function powerOfTwo() {                       // sample 2 at random, take the less loaded
  const a = servers[Math.floor(Math.random() * servers.length)];
  const b = servers[Math.floor(Math.random() * servers.length)];
  return a.conns <= b.conns ? a : b;
}
// On each request: s = pick(); s.conns++; ...forward...; on response: s.conns--
```

◆ **SENIOR INSIGHT** — **Why 'round robin' is often not enough**

- **Request cost varies by orders of magnitude** (a 5 ms health call vs a 2 s report). With plain round robin, an unlucky alternation can send every heavy request to the same server while others idle (see Example B in 7.12). Load-aware strategies (least-conn, P2C) adapt automatically.
- **Herd effect**: when a new or recovered server has 0 connections, 'least connections' sends it everything at once and it can fall over again. Use **slow start** (ramp weight over 30–120 s).
- **With many LB instances**, each has only a partial view of load; a purely global 'least loaded' choice by all of them synchronises them onto the same server. P2C's randomness avoids this — one reason it is popular in modern proxies and service meshes.
- Choose by workload: stateless short requests → least-conn/P2C; cache fleets or stateful shards → consistent hash; canary/blue-green → weighted.

7.5 Health checks, draining & failure handling

Load balancers are effective at **preventing requests from going to unhealthy servers** — but only if they can tell which are unhealthy.

Kind	How it works	Typical settings	Notes
Active (probe)	LB periodically calls each backend: TCP connect, or HTTP GET `/healthz` expecting 200	Interval 5–30 s, timeout 2–5 s, unhealthy after 2–3 failures, healthy after 2–5 successes	Detects dead hosts even with no traffic. Hysteresis (different thresholds) prevents flapping
Passive (outlier detection)	LB watches real traffic: connection errors, 5xx, timeouts → temporarily **ejects** the server	e.g., 5 consecutive 5xx → eject for 30 s (exponentially longer if repeated)	Reacts to real failures faster than probes; used by Envoy and service meshes

Shallow vs deep health checks

Check	Tests	Risk
Shallow / liveness	"Is the process up and able to answer?" (returns 200 immediately)	Misses a server that is up but cannot reach its DB
Deep / readiness	"Can I do useful work?" (pool connected, caches warmed, not shutting down)	If it checks a **shared** dependency (DB), one DB blip fails **every** server at once → LB has zero healthy targets → total outage

```
app.get('/healthz', (req, res) => res.status(200).send('ok'));      // liveness: process is alive

let shuttingDown = false;
app.get('/readyz', (req, res) =>                                      // readiness: take traffic?
  (shuttingDown || !localPoolHealthy()) ? res.status(503).end() : res.status(200).send('ready'));

process.on('SIGTERM', () => {            // deploy / scale-in: drain, don't drop
  shuttingDown = true;                   // 1) fail readiness so the LB stops sending new requests
  setTimeout(() => server.close(() => process.exit(0)), 10_000);   // 2) finish in-flight, then exit
});
```

★ **MUST REMEMBER** — **Connection draining (deregistration delay)**

- When a server is removed (deploy, scale-in, failed check), the LB stops sending **new** requests but lets **in-flight** ones finish (typically 30–300 s). Without draining, every deploy cuts live requests and users see 502/504 errors.

What the LB's error codes mean (recall Chapter 4)

Code	Meaning at the LB	Typical cause
502 Bad Gateway	Backend sent an invalid response or closed the connection unexpectedly	App crashed mid-request, keep-alive timeout mismatch (7.12 C), protocol mismatch
503 Service Unavailable	No healthy targets, or LB/backend queue is full (overload / load shedding)	All checks failing, capacity exhausted, bad deploy
504 Gateway Timeout	Backend didn't respond within the LB's timeout	Slow dependency, long query, backend overloaded

◆ **SENIOR INSIGHT** — **Retries at the LB: powerful and dangerous**

- An LB can retry a failed request on another backend — safe for **idempotent** requests or when the connection failed **before** any bytes were sent; unsafe for non-idempotent POSTs (double charge, Chapter 4).
- Always cap retries (1–2), use **retry budgets** (e.g., ≤ 10% extra traffic), per-try and overall timeouts, and add **circuit breaking / max pending requests** so overload turns into fast 503s rather than a growing queue (load shedding).

7.6 Sessions & sticky routing

If web apps do not keep track of sessions, the LB can provide **session persistence** (stickiness): issue a cookie and route a specific client's requests to the **same instance**.

Method	How	Pros	Cons
LB-issued cookie	LB sets a cookie (e.g., server id) and honours it	Per-user, survives IP changes	L7 only; cookie must be trusted/expiring
App-session cookie	LB reads the app's own session cookie	Matches app semantics	Needs L7 + config
Source-IP hash	Hash of client IP chooses the server	Works at L4, no cookies	Many users behind one NAT hit one server (Chapter 3); mobile IP changes break stickiness
Consistent hash on a key (user id, URL)	Key → server mapping	Cache locality, shard-aware routing	Rebalancing on membership change

✖ **COMMON MISTAKE** — **Stickiness is a crutch, not a design**

- Problems: uneven load (a few heavy users pin one server), **sessions lost when that server dies or scales in**, harder deploys, autoscaling that can't rebalance existing users.
- Better: make servers **stateless** (Chapter 5) and keep sessions in a central store — a database or persistent cache such as Redis/Memcached — or in signed tokens (JWT). Use stickiness only where state is inherent (WebSockets, in-memory game rooms, local caches for locality).

7.7 TLS, connection handling & preserving the client IP

Mode	What happens	When to use
SSL/TLS termination	LB decrypts incoming requests and encrypts server responses so backend servers do not have to perform these potentially expensive operations. **Removes the need to install X.509 certificates on each server**	Default for web apps; enables L7 routing, WAF, caching. Backend traffic is plain HTTP inside a trusted private network
Re-encryption (TLS to backends)	LB terminates then opens a new TLS (or mTLS) connection to the backend	Compliance (PCI, HIPAA), zero-trust networks, traffic crossing untrusted links
TLS passthrough	LB forwards encrypted bytes untouched (L4); backend holds the certificate	End-to-end encryption required; non-HTTP protocols

- **Certificates & SNI**: the LB serves many certificates on one IP, picking by the hostname in the TLS handshake (SNI). Automate issuance/renewal (ACM, Let's Encrypt) — expired certificates are a top cause of outages.
- **Protocol translation**: HTTP/2 or HTTP/3 to clients, HTTP/1.1 (or HTTP/2/gRPC) to backends. The LB **multiplexes** thousands of client connections onto a small pool of persistent backend connections, so servers hold far fewer sockets.
- **Client IP**: after a proxy, every connection arrives from the LB's IP. The LB adds `X-Forwarded-For` (client chain), `X-Forwarded-Proto`, `X-Forwarded-Host`; L4 LBs use the **PROXY protocol** or preserve source IP. **Trust these headers only from your own proxies** — otherwise clients can spoof them to dodge rate limits/IP bans.
- **Timeouts**: LB idle timeout, backend keep-alive timeout, request timeout, and client timeout must be ordered sensibly (see 7.12 Example C).

7.8 Reverse proxy (and API gateway)

A **reverse proxy** is a web server that **centralizes internal services and provides unified interfaces to the public**. Requests from clients are forwarded to a server that can fulfil them, and the reverse proxy returns the server's response to the client. (A *forward* proxy sits in front of clients; a *reverse* proxy sits in front of servers — clients only ever see the proxy.)

```
   (public)                    (private network: example.com)
Internet  <---->  [ Reverse proxy ]  <---->  Web server / app servers / other services
                  (one public IP)
```

Benefits

Benefit	Details
Increased security	Hide information about backend servers (IPs, versions), **blacklist IPs**, **limit the number of connections per client**, WAF rules, a single hardened entry point
Increased scalability and flexibility	Clients only see the reverse proxy's IP, so you can scale servers or change their configuration freely
SSL termination	Decrypt incoming requests and encrypt server responses so backend servers don't do this expensive work; removes the need to install X.509 certificates on each server
Compression	Compress server responses (gzip/Brotli) once at the edge
Caching	Return the response for cached requests without contacting the backend
Static content	Serve HTML/CSS/JS, photos, videos etc. directly from disk, freeing app servers

Example NGINX configuration (reverse proxy + load balancer)

```
upstream app_backend {
    least_conn;                                  # algorithm (default: round robin)
    server 10.0.1.11:3000 max_fails=3 fail_timeout=10s;     # passive health check
    server 10.0.1.12:3000 weight=2;              # gets twice the share
    server 10.0.1.13:3000 backup;                # used only if the others are down
    keepalive 64;                                # pool of persistent upstream connections
}
server {
    listen 443 ssl http2;                        # TLS termination + HTTP/2 to clients
    server_name api.example.com;
    ssl_certificate     /etc/ssl/api.crt;
    ssl_certificate_key /etc/ssl/api.key;

    location /static/ { root /var/www; expires 1y; add_header Cache-Control "public, immutable"; }

    location / {
        proxy_pass http://app_backend;
        proxy_http_version 1.1;  proxy_set_header Connection "";        # needed for keepalive
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 2s;  proxy_read_timeout 30s;               # fail fast, bound waits
    }
}
```

Load balancer vs reverse proxy

	Load balancer	Reverse proxy
Main purpose	Spread traffic over **multiple servers**, avoid unhealthy ones	Be the single front door: security, TLS, caching, compression, routing
When it pays off	**When you have multiple servers**; often routing traffic to a set serving the same function	**Even with just one web or application server** — all the benefits above apply
Overlap	Solutions such as **NGINX and HAProxy support both layer 7 reverse proxying and load balancing** — in practice one box often plays both roles	

Reverse proxy vs API gateway

	Reverse proxy / L7 LB	API gateway
Focus	Traffic management and protection	**API management** as a product: auth, quotas, developer keys
Typical features	Routing, TLS, caching, compression, load balancing, basic rate limiting	Authentication/authorisation (JWT/OAuth), per-client **rate limiting & quotas**, request/response transformation, API versioning, aggregation, analytics, developer portal
Examples	NGINX, HAProxy, Envoy, cloud LBs	Kong, Apigee, AWS API Gateway, Azure API Management, NGINX/Envoy-based gateways

They overlap and are often layered: **CDN → WAF/LB → API gateway → services**. Chapter 8 returns to API gateways and service meshes.

7.9 Making the load balancer itself highly available

A load balancer removes the server tier as a single point of failure — but **a single load balancer is itself a single point of failure**. Configuring multiple load balancers further increases complexity. To protect against failures it is common to set up multiple load balancers, either in **active-passive** or **active-active** mode (Chapter 5).

Approach	How	Notes
Active-passive pair	Two LBs share a **virtual/floating IP** via VRRP (keepalived); heartbeat failure → standby takes over the VIP	Simple; idle capacity; failover in seconds; connection state may be lost unless synchronised
Active-active via DNS	Several LBs, each with its own IP, all returned by DNS (Chapter 6)	Uses all capacity; DNS caching slows failover; each LB needs a health check
Active-active via ECMP / anycast	Routers spread flows across many L4 LBs using ECMP; the same VIP is announced by all (BGP). Consistent hashing keeps a connection on one LB even when the fleet changes (e.g., Google's Maglev)	How hyperscale L4 balancing works; needs network expertise
Cloud managed LB	Provider runs a fleet of LB nodes across AZs behind a DNS name and scales them automatically	Easiest. Very sudden spikes may need provider pre-warming; cross-zone balancing setting affects even distribution

◆ **SENIOR INSIGHT** — **Capacity, not just uptime**

- The load balancer **can become a performance bottleneck** if it does not have enough resources (CPU for TLS, network bandwidth, connection table, file descriptors) or if it is not configured properly. Monitor active connections, new connections/s, TLS handshakes/s, bandwidth, queue depth, 5xx rate, target response time.
- **Blast radius**: one mis-configured LB rule can take down everything behind it. Treat LB config as code: review, canary, automated rollback, and per-environment separation.
- **Global vs regional**: DNS/anycast/global LB chooses a region; regional LBs choose zones; zonal targets choose instances. **Cross-zone balancing** evens out load when zones have different numbers of servers but may add cross-AZ network cost.

7.10 Horizontal scaling with a load balancer

Load balancers can also help with **horizontal scaling**, improving performance and availability. Scaling out using commodity machines is more cost efficient and results in higher availability than scaling up a single server on more expensive hardware (vertical scaling). It is also easier to hire for talent working on commodity hardware than for specialised enterprise systems.

Disadvantage of horizontal scaling	Mitigation
Scaling horizontally introduces complexity and involves **cloning servers**	Immutable images, containers, configuration management, autoscaling groups
Servers should be **stateless**: they should not contain any user-related data like sessions or profile pictures	Sessions in a centralized data store (database SQL/NoSQL) or a persistent cache (Redis, Memcached); files in object storage
Downstream servers such as **caches and databases need to handle more simultaneous connections** as upstream servers scale out	Connection pooling and poolers/proxies (PgBouncer, ProxySQL), read replicas, caching, limit per-instance pool size (total = instances × pool)

Autoscaling

- **Scale on the right signal**: CPU is a rough proxy; better are **requests per target**, queue depth, or latency (SLO-based). Use **target tracking** ("keep ~60% CPU" or "500 req/s per instance").
- **Warm-up & cooldown**: new instances need time (boot, cache warm-up, JIT); give the LB a slow-start period and don't flap with rapid in/out cycles.
- **Scale-in safely**: drain connections first (7.5); never terminate instances holding unreplicated state.
- **Scale ahead of known events** (scheduled scaling) — reactive autoscaling lags by minutes.

Capacity planning with failure in mind

```
servers needed = peak QPS / (per-server max QPS x target utilisation)
Survive the loss of one of N zones:  each zone must run at <= (N-1)/N of capacity
    2 zones -> <= 50% utilised each          3 zones -> <= 67%          4 zones -> <= 75%
```

7.11 Disadvantages & common pitfalls

Problem	Why it happens	Mitigation
LB becomes a bottleneck	Not enough resources or not configured properly (TLS CPU, bandwidth, connection limits)	Right-size, scale out LB fleet, offload TLS, monitor saturation
Added complexity	Introducing a load balancer to eliminate a single point of failure results in increased complexity	Managed LBs, infrastructure-as-code, runbooks
LB is a SPOF	A single load balancer is a single point of failure; configuring multiple load balancers further increases complexity	Active–passive / active–active LBs (7.9) or managed multi-AZ service
Uneven load	Round robin with variable request costs; sticky sessions; long-lived connections (WebSocket) pinned to old servers	Least-conn / P2C, avoid stickiness, rebalance by recycling connections periodically
Thundering herd after restart/scale-out	New servers get everything at once; clients reconnect simultaneously	Slow start, jittered reconnects, gradual rollouts
Timeout mismatch → sporadic 502s	Backend closes idle connections before the LB does	Backend keep-alive timeout > LB idle timeout (7.12 C)
Lost client IP	Backends see the LB's address	X-Forwarded-For / PROXY protocol; trust only known proxies
Health-check mistakes	Checks too shallow (route traffic to broken servers) or too deep (a shared DB hiccup removes every server)	Liveness vs readiness; hysteresis; 'fail open' when all targets look unhealthy
Retry storms	Retries at LB + client + service multiply load	Retry budgets, one retry layer, backoff + jitter (Chapter 4)

7.12 Worked examples

Example A — Sizing a stateless tier across three zones (peak 12,000 QPS; each server benchmarked at 1,000 QPS max; keep ≤ 80% utilisation even when one zone is down)

```
Capacity needed with one zone lost (2 of 3 zones left):  12,000 / (1,000 x 0.80) = 15 servers in 2 zones
Total servers = 15 x 3/2 = 22.5  ->  24 servers = 8 per zone
Check normal operation: 12,000 / 24,000 = 50% utilisation.   Zone loss: 12,000 / 16,000 = 75%  (OK, < 80%)
Cost of 'zone-proof' redundancy: 24 servers instead of the 12-15 you'd need with no failure budget.
```

Example B — Round robin vs least connections (2 servers; requests alternate heavy 2 s, light 10 ms, heavy, light …)

	Round robin	Least connections
Routing	R1 (heavy) → S1, R2 (light) → S2, R3 (heavy) → S1, R4 (light) → S2 …	R1 → S1; R2 → S2; R3 arrives while S1 still busy with R1 → goes to S2
Outcome	**S1 receives every heavy request** (queueing, ~4 s of work, high latency); S2 sits almost idle	Heavy requests spread across both servers; queues stay short, p99 latency much lower
Lesson	RR assumes equal cost per request	Load-aware choice adapts to uneven cost

Example C — The classic sporadic 502 (keep-alive timeout mismatch)

An AWS ALB keeps idle connections to targets for **60 s** and reuses them. A Node.js server closes idle keep-alive sockets after only **5 s** by default (`server.keepAliveTimeout = 5000`). When the LB sends a request on a connection the server has just closed, the request fails with ECONNRESET and the client sees a random **502** — rare, load-dependent, and hard to reproduce.

```
const server = app.listen(3000);
server.keepAliveTimeout = 65_000;   // must be GREATER than the LB idle timeout (ALB default 60 s)
server.headersTimeout   = 66_000;   // must be greater than keepAliveTimeout
// Rule: backend keep-alive timeout > load balancer idle timeout > client's idle timeout
```

7.13 Practice questions (try before looking)

P1. Users report a random 502 once in a few thousand requests behind an ALB with Node backends. First suspect?

Answer: Keep-alive timeout mismatch: the Node server (5 s default) closes idle connections before the LB (60 s). Set `keepAliveTimeout` above the LB idle timeout.

P2. Your health check queries the database. The DB has a 20-second blip. What can happen?

Answer: Every server fails its check at once, the LB has no healthy targets and returns 503 even after the DB recovers briefly. Use shallow checks for the LB (or fail-open) and handle DB errors in the app.

P3. Choose an algorithm: (a) identical stateless API servers, (b) a fleet of cache nodes where the same URL should hit the same node, (c) servers of different sizes.

Answer: (a) Least connections or P2C. (b) Consistent hashing on the URL. (c) Weighted round robin / weighted least connections.

P4. Peak 9,000 QPS, server max 750 QPS, ≤ 70% utilisation, must survive losing 1 of 3 zones. Servers needed?

Answer: Capacity with 2 zones: 9,000/(750×0.7) = 17.1 → 18 servers in two zones; total = 18 × 3/2 = **27** (9 per zone).

P5. Why not just use IP-hash for sticky sessions everywhere?

Answer: Clients behind one NAT/proxy (offices, mobile carriers) all hash to one server (hot spot), IPs change on mobile networks, and a dead server still loses its sessions. Prefer stateless servers + a shared session store.

Interview Q&A (30-second answers)

Q1. What does a load balancer do and why do we need one?

Distributes requests across servers, avoids unhealthy ones via health checks, gives clients a single stable endpoint — enabling horizontal scaling and removing the server tier as a SPOF.

Q2. L4 vs L7 load balancing?

L4 routes on IPs/ports without reading packet contents (NAT, fast, flexible protocols, low CPU). L7 terminates the connection, reads HTTP, routes by path/host/header/cookie and can rewrite, cache, authenticate — more flexible, more CPU.

Q3. Name load-balancing algorithms and when you'd use each.

Round robin (identical, similar requests), weighted (mixed capacity/canary), least connections (variable duration), IP/consistent hash (affinity, cache locality), power-of-two-choices (large fleets, distributed LBs).

Q4. How does the LB know a server is down?

Active health checks (TCP/HTTP probes with thresholds) and passive outlier detection on real traffic errors; removed servers are drained gracefully and re-added after consecutive successes.

Q5. What are sticky sessions and what are the alternatives?

Routing a client to the same instance via cookie or IP hash. Alternatives: stateless servers with sessions in Redis/DB or signed tokens; keep stickiness only for inherently stateful connections.

Q6. What is SSL termination and why do it at the LB/proxy?

Decrypt at the edge so backends skip expensive crypto and don't each need certificates; enables L7 routing and WAF. Re-encrypt to backends if compliance requires.

Q7. Load balancer vs reverse proxy vs API gateway?

LB spreads load over many servers; a reverse proxy fronts one or many servers for security/TLS/caching/compression; an API gateway adds API management (auth, quotas, transformation). NGINX/HAProxy/Envoy can play the first two roles; they are often layered.

Q8. How do you avoid the load balancer being a single point of failure?

Redundant LBs: active–passive with a floating IP (VRRP), active–active via DNS or ECMP/anycast, or a managed multi-AZ cloud LB; plus capacity monitoring.

Q9. How do you preserve the client's IP behind a proxy?

X-Forwarded-For (L7) or PROXY protocol (L4); trust these only from your own proxies to prevent spoofing.

Q10. How do you deploy without dropping requests?

Readiness fails first, LB stops new requests, connection draining lets in-flight finish, then the instance exits; roll out gradually (rolling/canary/blue-green with weighted routing).

Chapter 7 Cheat Sheet

⚡ **QUICK RECALL** — **Revise this in 2 minutes**

- **LB** = distribute + health-check + single entry point. Prevents overload and unhealthy targets; removes server-tier SPOF but **must itself be redundant**.
- **Types:** hardware (F5, costly) • software (HAProxy, NGINX, Envoy) • cloud managed (ALB L7, NLB L4) • client-side • mesh sidecar • DNS.
- **L4** = IP+port, no content, NAT, fastest, any protocol. **L7** = terminates, reads HTTP, path/host/cookie routing, rewrite/auth/cache, more CPU. Often L4 in front of L7.
- **Algorithms:** random • round robin • weighted • **least connections** • least response time • **IP/consistent hash** • **power of two choices**. RR fails with uneven request cost.
- **Health:** active probes + passive ejection; liveness vs readiness; shallow beats deep for LB checks; slow start; **connection draining**.
- **Sessions:** prefer stateless + Redis/DB; sticky (cookie/IP hash) = crutch with uneven load and lost sessions.
- **TLS:** terminate at LB (no certs on each server) / re-encrypt / passthrough. Keep client IP via **X-Forwarded-For** or PROXY protocol — trust only your proxies.
- **Reverse proxy** = front door: security (hide backends, blacklist IPs, limit connections), scalability/flexibility, TLS, compression, caching, static files; useful even with one server.
- **API gateway** = reverse proxy + API management (auth, quotas, transformation).
- **HA of LB:** VRRP floating IP (active-passive) • DNS or ECMP/anycast (active-active) • managed multi-AZ.
- **Capacity:** servers = peak / (max × target util); survive 1 of N zones → run each at ≤ (N−1)/N. Backend keep-alive > LB idle timeout to avoid 502s.
- **Horizontal scaling costs:** cloning/complexity, stateless servers, more connections to caches/DBs (pool them).

AMJESH https://singhamjesh.github.io/